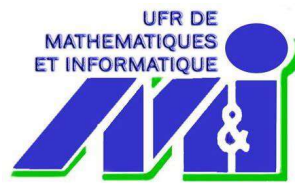




Ministère de l'Enseignement
Supérieur et de la
Recherche Scientifique

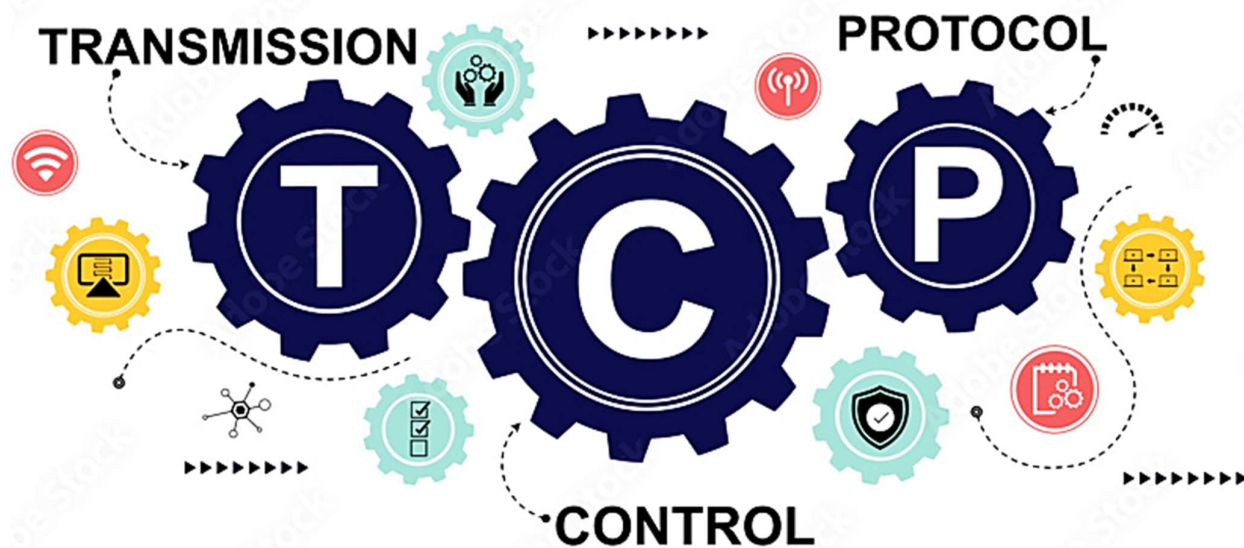


Année Universitaire 2025 – 2026

Licence 3 – Réseaux Informatique Sécurité et Télécommunications

PROJET DE FIN DE MODULE — GESTION DE PROJET RESEAUX

INGENIERIE DES PROTOCOLES : ETUDE COMPARATIVE DES VERSIONS TCP NEW-RENO, BBR ET CUBIC



Produit par :

AGBENONZAN Kossivi Jacques Junior

SANOGO Souleymane

Encadreur : Dr DIALLO

Table des matières

INTRODUCTION.....	2
A. Environnement technique.....	3
B. Analyse des Protocoles TCP	4
1. BBR	4
1.1. Qu'est ce que TCP BBR ?	4
1.2. Analyse de performance de TCP BBR-4-0	6
1.3. Conclusion sur TCP BBR.....	9
2. TCP New Reno.....	10
2.1. Qu'est-ce que TCP NewReno ?	10
2.2. Analyse de performance de TCP NewReno.....	11
2.3. Conclusion sur TCP NewReno	13
3. TCP CUBIC	14
3.1. Qu'est-ce que TCP CUBIC ?	14
3.2. Analyse de performance de TCP CUBIC	14
3.3. Conclusion sur TCP CUBIC	17
4. Analyse Comparative des Protocoles TCP	17
4.1. Tableau comparatif des métriques principale	18
4.2. Synthèse et interprétation des différences	19
4.3. Limites	19
4.4. Recommandations	20
CONCLUSION GÉNÉRALE.....	21
Perspectives.....	22
Liste des mots techniques et termes compliqués	23
Contenue du Script de configuration et récupération du Trafic TCP	24
Contenue du Script de lancement	28

INTRODUCTION

Dans le paysage actuel des réseaux informatiques, le protocole TCP (*Transmission Control Protocol*) demeure la pierre angulaire des communications fiables sur Internet. Son évolution constante a donné naissance à plusieurs variantes, chacune proposant des mécanismes distincts pour optimiser le contrôle de congestion et améliorer les performances.

Dans le cadre de notre formation à l'Université Félix Houphouët-Boigny, le projet de fin de module en **Gestion de Projet Réseaux** vise à mettre en pratique les concepts théoriques étudiés sur les protocoles de transport et leur comportement dans un environnement simulé.

Ce projet a pour objectif de comparer les performances de trois versions majeures du protocole TCP : **TCP NewReno**, **TCP CUBIC** et **TCP BBR**. Chacune de ces variantes adopte une approche différente du contrôle de congestion :

- **TCP NewReno**, successeur de TCP Reno, reste une référence historique grâce à sa gestion classique des pertes par ajustement de la fenêtre de congestion.
- **TCP CUBIC**, devenu l'algorithme par défaut dans le noyau Linux, se distingue par sa fonction de croissance cubique indépendante du RTT, particulièrement adaptée aux réseaux à large bande passante.
- **TCP BBR** (*Bottleneck Bandwidth and Round-Trip Time*), développé par Google, repose sur l'estimation du goulot d'étranglement plutôt que sur la détection des pertes de paquets.

L'objectif de cette étude est d'analyser et de quantifier les différences de comportement de ces trois protocoles en termes de **débit, délai, pertes de paquets** et **équité**, à travers des simulations réalisées avec l'outil NS-3 (*Network Simulator 3*).

Ce rapport présente successivement la méthodologie adoptée pour la conception des scénarios de simulation, l'implémentation technique sous NS-3, l'analyse détaillée des résultats obtenus pour chaque variante TCP, ainsi qu'une comparaison synthétique avant de conclure sur les forces et faiblesses respectives de NewReno, CUBIC et BBR dans le contexte de notre étude.

A. Environnement technique

Le projet a été réalisé sur deux configurations matérielles distinctes :

Machine 1 (HP ProBook 440 G5 – Machine AGBENONZAN)

- **Processeur** : Intel Core i5-8250U @ 1,60 GHz (Turbo à 1,80 GHz)
- **Mémoire RAM** : 8 Go
- **Système hôte** : Windows 11 Professionnel Insider Preview (build 26300.8068)
- **Environnement de simulation** : WSL2 (Windows Subsystem for Linux 2) avec Ubuntu 24.04.1 LTS
- **Version de NS-3** : 3.40

Machine 2 (HP ProBook 640 G1 – Machine Sanogo)

- **Processeur** : Intel Core i5-4200M @ 2,50 GHz
- **Mémoire RAM** : 12 Go
- **Système hôte** : Windows 10 Professionnel (build 19045)
- **Environnement de simulation** : MSYS2 MINGW64
- **Version de NS-3** : 3.40

Cette diversité d'environnements (WSL2 sous Ubuntu versus MSYS2 sous Windows) a permis de valider la **portabilité** des simulations ainsi que la **reproductibilité** des résultats sur différentes plateformes.

Outils utilisés

- **NS-3.40** : simulateur réseau à événements discrets
- **C++** : langage de développement des scripts de simulation
- **Wireshark** : analyse des captures de paquets (fichiers PCAP)
- **Bash** : automatisation des scripts d'exécution

B. Analyse des Protocoles TCP

```
=====
COMPARAISON DES PROTOCOLES TCP
NewReno vs CUBIC vs BBR
=====

[1/4] Nettoyage des anciens fichiers...
[2/4] Compilation en cours...
Finished executing the following commands:
cd cmake-cache; /usr/bin/cmake --build . -j 7 --target scratch_comparison-tcp_tcp-comparison ; cd ..
[3/4] Exécution des simulations...
Simulation en cours... (15 secondes)

=== COMPARAISON DES PROTOCOLES ===
=====

NEWRENO (10.1.1.1)
Débit : 1.14 Mbps
Pertes : 9.00 paquets
Tx : 3449.00 paquets
Rx : 3404.00 paquets
Efficacité: 98.70%
-----

CUBIC (10.1.2.1)
Débit : 1.49 Mbps
Pertes : 61.00 paquets
Tx : 4538.00 paquets
Rx : 4438.00 paquets
Efficacité: 97.80%
-----

BBR (10.1.3.1)
Débit : 1.44 Mbps
Pertes : 19.00 paquets
Tx : 4346.00 paquets
Rx : 4282.00 paquets
Efficacité: 98.53%
-----

■ Résultats sauvegardés dans 'resultats_comparaison.csv'
[4/4] Résultats obtenus:
ERREUR: Fichier resultats_comparaison.csv non trouve

=====
EXPÉRIENCE TERMINÉE
=====
```

Fig n°0 : Lancement du script de lancement du fichier de configuration

1. BBR

1.1. Qu'est ce que TCP BBR ?

TCP BBR (*Bottleneck Bandwidth and Round-Trip Time*) est un algorithme de contrôle de congestion développé par Google. Contrairement aux approches classiques comme TCP NewReno ou TCP CUBIC, qui se basent principalement sur la détection des pertes de paquets pour réduire la fenêtre de congestion, BBR adopte une approche proactive basée sur un modèle du réseau.

Il estime en continu deux paramètres clés du chemin réseau :

- **Bottleneck Bandwidth (BtlBw)** : le débit maximal disponible au goulot d'étranglement ;

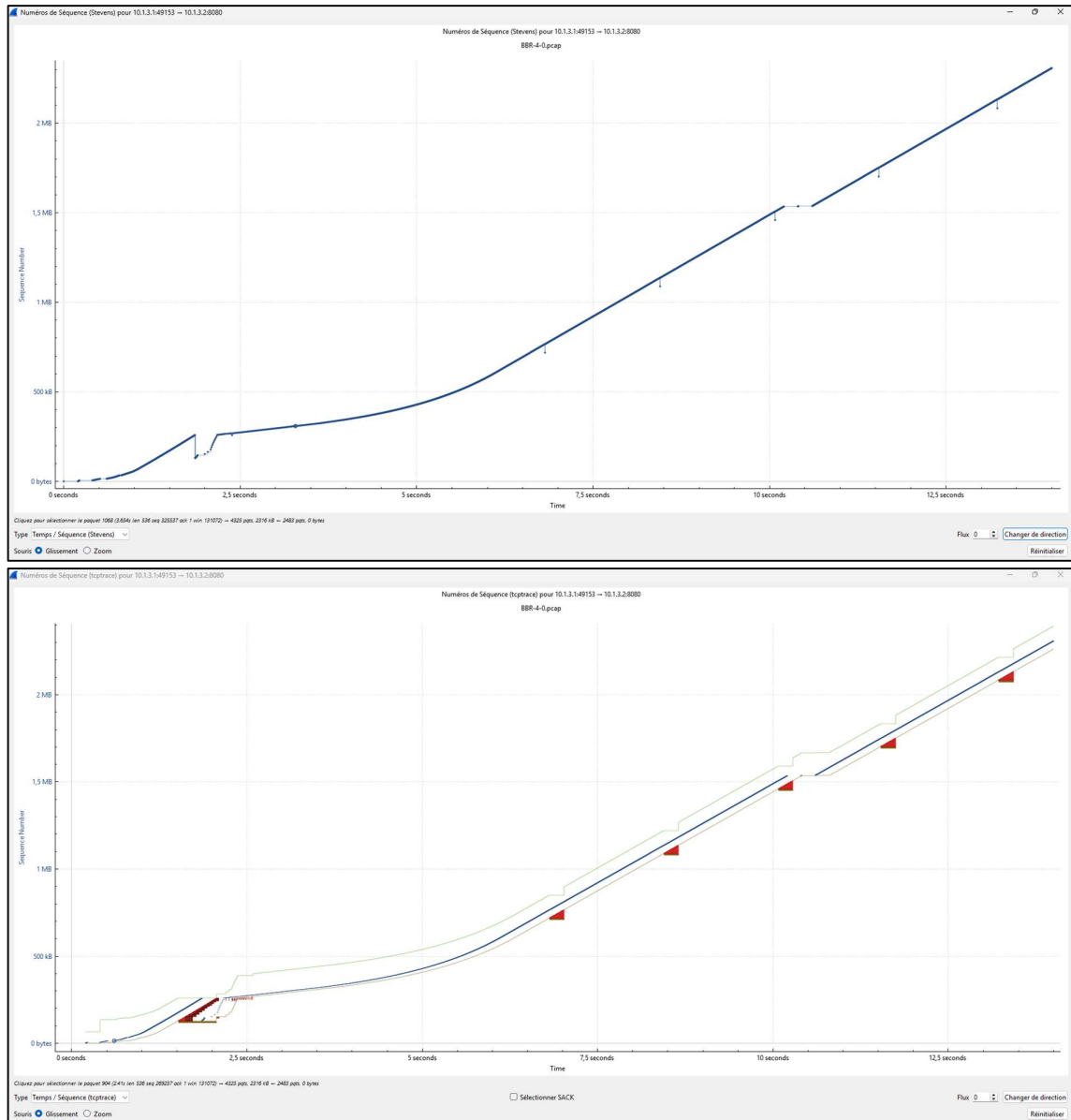
- **Round-Trip propagation time minimal (RTprop)** : le délai de propagation minimal en l'absence de file d'attente.

L'objectif de BBR est d'opérer au point optimal où le débit d'envoi correspond exactement à la capacité du lien, tout en maintenant le nombre de paquets en vol proche du *Bandwidth-Delay Product* (BDP). Cette stratégie permet de minimiser la formation de files d'attente et les pertes inutiles.

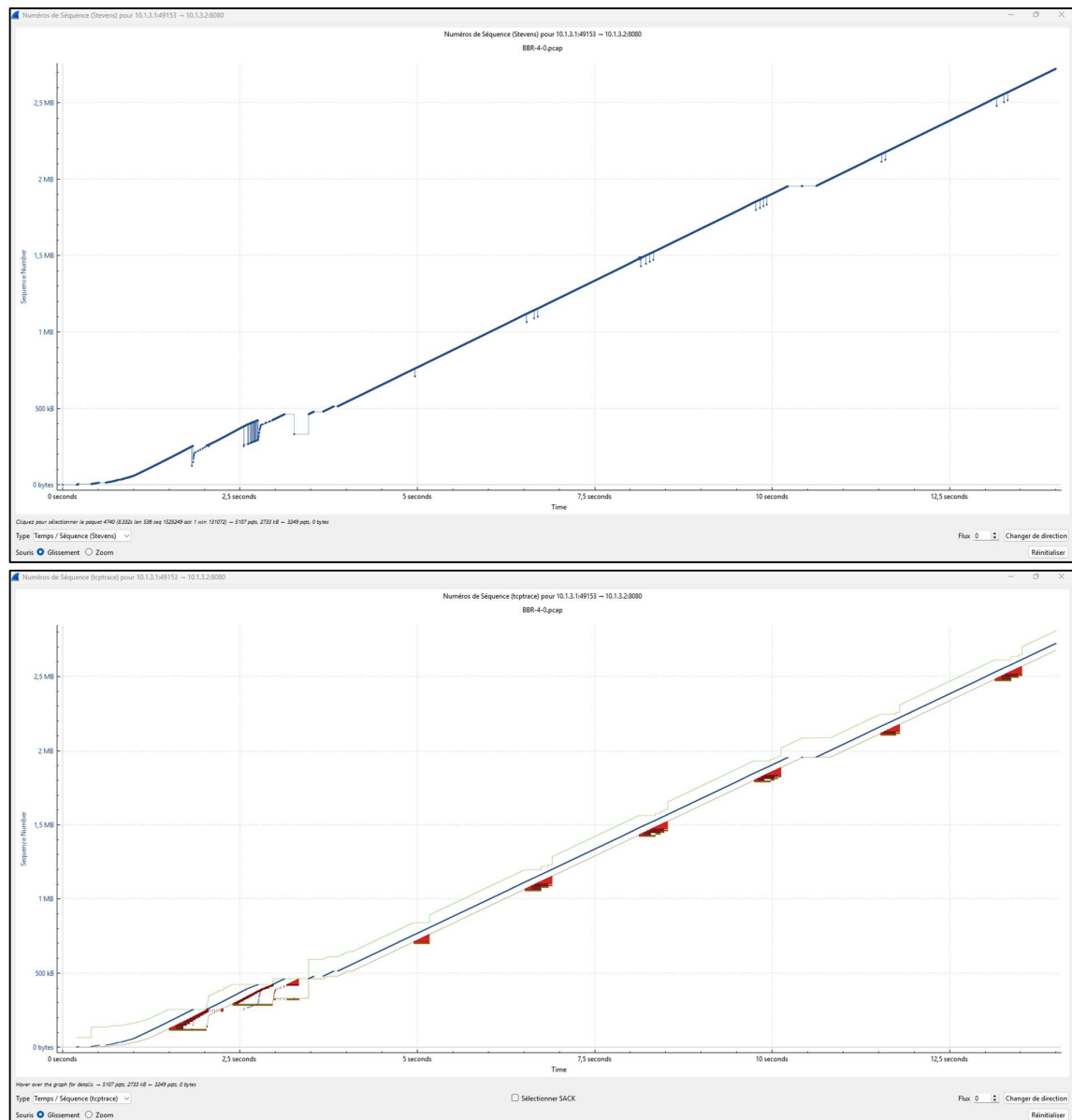
Le fonctionnement de TCP BBR se décompose en quatre phases principales :

- **Startup** : phase initiale agressive durant laquelle BBR augmente rapidement le *pacing rate* pour découvrir la bande passante maximale ;
- **Drain** : réduction temporaire du débit afin de vider les files d'attente accumulées ;
- **ProbeBW** : phase de régime stable où BBR explore périodiquement la bande passante via un cycle de gains de *pacing* , tout en maintenant la fenêtre de congestion autour de $2 \times \text{BDP}$;
- **ProbeRTT** : phase périodique où BBR réduit temporairement le volume de données en transit pour mesurer un RTprop plus précis.

1.2. Analyse de performance de TCP BBR-4-0



Figs: Capture WireShark, Graphes TCP BBR(Stevens et tcptrace) Machine AGBENONZAN



Figs: Capture WireShark, Graphes TCP BBR(Stevens et tcptrace) Machine SANOGO

L'analyse des captures Wireshark issues des simulations NS-3.40 (fichiers BBR-0-0.pcap sur les deux machines) révèle un comportement caractéristique et parfaitement reproductible de TCP BBR, malgré les différences d'environnement (WSL2 Ubuntu sur la machine AGBENONZAN versus MSYS2 MINGW64 sur la machine SANOGO).

- **Établissement de la connexion (paquets 1 à 3)**

L'analyse de la capture réalisée sur la machine SANOGO montre un *handshake* TCP classique. La fenêtre annoncée passe de 65 535 octets dans les paquets SYN et SYN-ACK à **131 072 octets** dès le troisième paquet (premier ACK du client).

5	0.203643	10.1.3.1	10.1.3.2	TCP	542 49153 → 8080 [ACK] Seq=537 Ack=1 Win=131072 Len=48
6	0.205811	10.1.3.1	10.1.3.2	TCP	590 49153 → 8080 [ACK] Seq=1025 Ack=1 Win=131072 Len=5
7	0.208171	10.1.3.1	10.1.3.2	TCP	542 49153 → 8080 [ACK] Seq=1561 Ack=1 Win=131072 Len=4

Cette valeur de fenêtre reste **constante** tout au long de la simulation. Un comportement identique a été observé sur la capture de la machine AGBENONZAN.

- **Phase Startup (≈ 0 à 3 s)**

Dès le début de la transmission de données, on observe un envoi agressif et soutenu. Sur la machine SANOGO, les paquets se succèdent avec des intervalles très courts. Le graphe d'évolution du numéro de séquence présente une pente raide initiale, traduisant une phase d'exploration rapide de la bande passante.

Cette agressivité est conforme à la phase *Startup* de BBR. Une légère irrégularité apparaît entre 1,5 s et 3 s, correspondant au passage vers les phases *Drain* puis *ProbeBW*. Des pertes mineures sont détectées via des ACK dupliqués et des options SACK, entraînant des retransmissions rapides isolées, sans toutefois provoquer d'effondrement de la fenêtre.

- **Phase ProbeBW (≈ 3 s à 15 s)**

Après la phase transitoire, la courbe du numéro de séquence devient **quasi-linéaire et extrêmement régulière** sur les deux captures. Le débit est stable et soutenu, avec très peu d'oscillations. La fenêtre reste bloquée à 131 072 octets, ce qui correspond à un fonctionnement en *ProbeBW* où BBR maintient un envoi proche de la capacité estimée du lien (2 Mbps).

Le délai de propagation configuré (100 ms) et la file d'attente limitée (15 paquets) permettent de contenir le *bufferbloat*. BBR opère ainsi avec une latence maîtrisée et un faible taux de pertes. Quelques périodes d'ACK dupliqués apparaissent en fin de simulation, mais sont gérées efficacement sans chute brutale du débit.

✓ **Observations communes aux deux machines**

- Stabilité du débit après environ 3 s ;

- Fenêtre de congestion constante à 131 072 octets ;
- Pertes modérées, gérées par retransmissions rapides et SACK ;
- Comportement parfaitement reproductible sur WSL2 et MSYS2.

Les graphiques TCP (Stevens et tcptrace) générés à partir des deux PCAP montrent une courbe lisse et stable en régime *ProbeBW*, contrastant nettement avec les oscillations typiques des protocoles *loss-based*.

1.3. Conclusion sur TCP BBR

TCP BBR représente une évolution majeure des mécanismes de contrôle de congestion. Dans le contexte précis de notre simulation (lien bottleneck de 2 Mbps, délai de propagation de 100 ms et file d'attente limitée à 15 paquets), il démontre une **excellente stabilité du débit**, une utilisation très efficace de la bande passante et une réduction significative des files d'attente (*bufferbloat*).

Contrairement aux approches traditionnelles *loss-based* qui interprètent toute perte comme un signe de congestion, BBR maintient un envoi régulier et soutenu, même en présence de pertes modérées. Sa phase *Startup* agressive peut toutefois engendrer quelques pertes initiales. Par ailleurs, son comportement en termes d'équité face à d'autres flux concurrents reste un sujet d'étude important dans les environnements multi-flux.

L'analyse des captures Wireshark réalisées sur les deux machines (AGBENONZAN sous WSL2 et SANOGO sous MSYS2) confirme la **grande reproductibilité** des résultats et la parfaite cohérence de l'implémentation de TCP BBR dans NS-3.40.

2. TCP New Reno

2.1. Qu'est-ce que TCP NewReno ?

TCP NewReno est une amélioration de TCP Reno et fait partie des algorithmes de contrôle de congestion *loss-based* (basés sur les pertes). Il est considéré comme une référence historique et reste largement utilisé grâce à sa simplicité et sa robustesse.

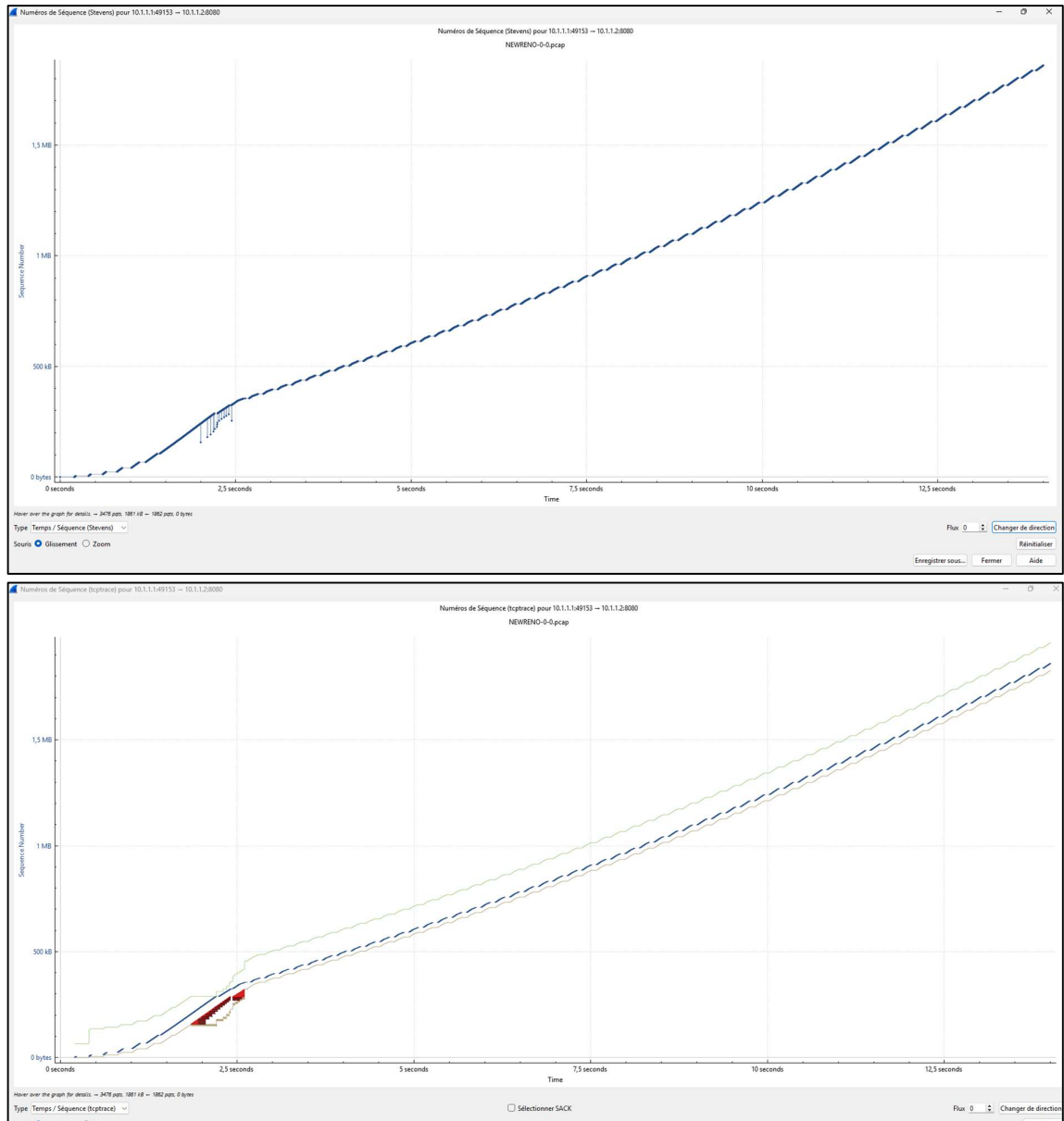
NewReno ajuste principalement sa fenêtre de congestion en réaction à la détection de pertes, que ce soit via les ACK dupliqués ou les expirations de temporisateur (*timeouts*).

Ses mécanismes principaux sont :

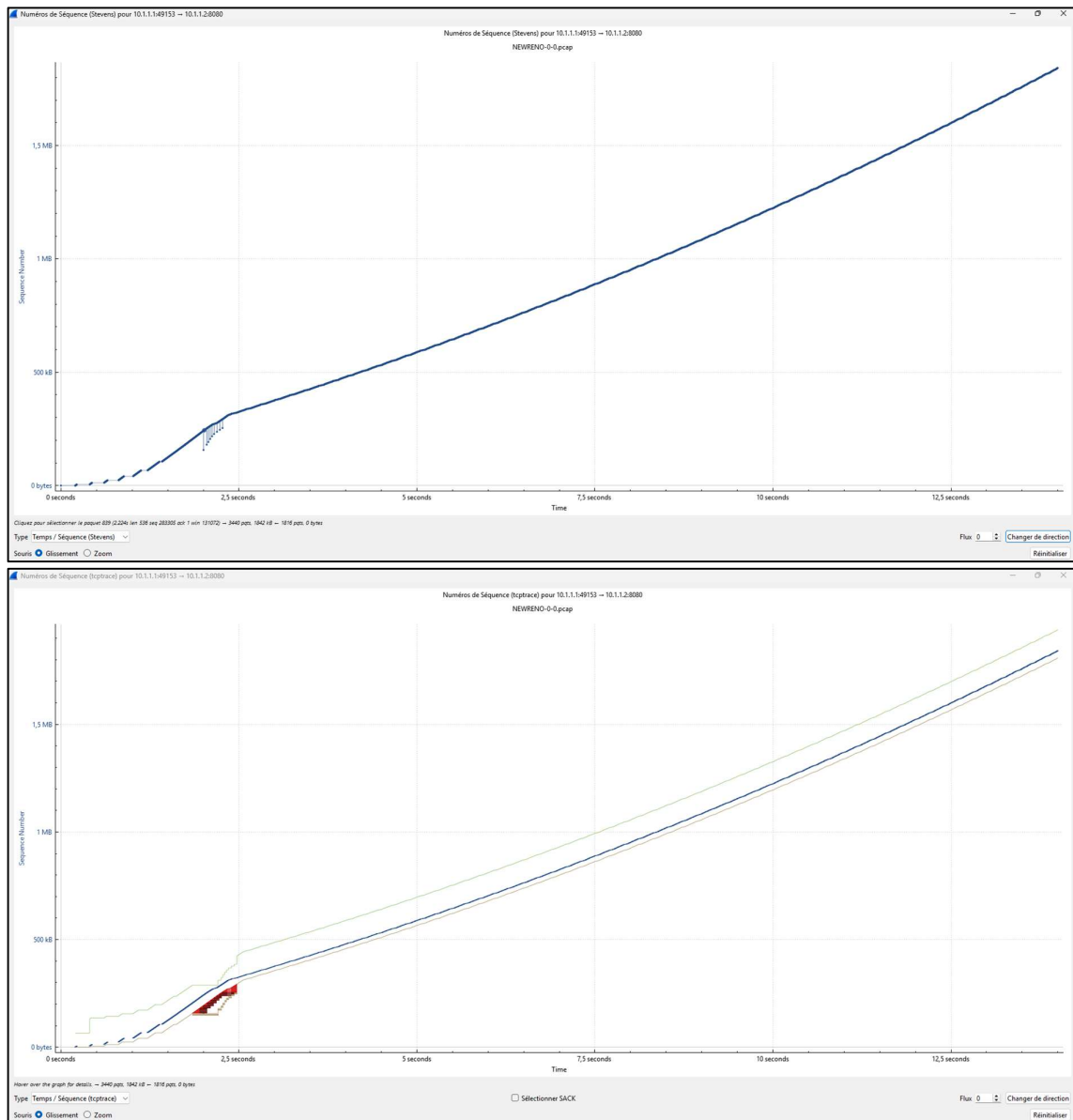
- **Slow Start** : croissance exponentielle de la fenêtre de congestion au début de la connexion ;
- **Congestion Avoidance** : croissance linéaire une fois le seuil de Slow Start atteint ;
- **Fast Retransmit & Fast Recovery** : retransmission rapide en cas de réception de trois ACK dupliqués ;
- **Amélioration NewReno** : meilleure gestion des pertes multiples grâce à la récupération partielle (*partial ACK*).

Contrairement à BBR qui s'appuie sur un modèle du réseau, NewReno adopte une approche **réactive** aux pertes : il réduit fortement sa fenêtre de congestion (généralement par deux) dès qu'une congestion est détectée.

2.2. Analyse de performance de TCP NewReno



Figs: Capture WireShark, Graphes TCP NewReno(Stevens et tcptrace) Machine SANOGO



Figs: Capture WireShark, Graphes TCP NewReno(Stevens et tcptrace) Machine AGBENONZAN

L'analyse de la capture PCAP de TCP NewReno sur la machine SANOGO révèle un comportement typique des algorithmes *loss-based* classiques dans un environnement contraint : bande passante limitée à 2 Mbps et délai de propagation de 100 ms.

- **Établissement de la connexion**

Le *handshake* TCP est classique. La fenêtre annoncée passe de 65 535 octets à **131 072 octets** dès le troisième paquet.

Protocol	Length	Info
TCP	58	49153 → 8080 [SYN] Seq=0 Win=65535 Len=0 TSval=100
TCP	58	8080 → 49153 [SYN, ACK] Seq=0 Ack=1 Win=65535 Len=
TCP	54	49153 → 8080 [ACK] Seq=1 Ack=1 Win=131072 Len=0 TS

Contrairement à BBR où cette valeur reste fixe, la fenêtre de NewReno **évolue** tout au long de la simulation.

- **Phase initiale (Slow Start)**

Dès les premiers paquets de données, on observe un envoi rapide avec des paquets de 536 octets. La courbe du numéro de séquence présente une pente initiale très raide, caractéristique de la phase Slow Start où la fenêtre double à chaque RTT.

- **Comportement en régime stable (Congestion Avoidance)**

Après la phase de démarrage, NewReno passe en Congestion Avoidance. La courbe du numéro de séquence montre une croissance linéaire, mais avec une pente nettement plus faible que celle observée sous BBR. Des périodes de transmission régulières sont entrecoupées d'ACK dupliqués, signe de pertes de paquets.

Plusieurs événements de **Fast Retransmit** apparaissent tout au long de la trace. À chaque détection de congestion, NewReno réduit sa fenêtre de moitié, puis reprend une croissance linéaire. Ce comportement se traduit par des **oscillations en dents de scie** visibles sur les graphes de la fenêtre de congestion et du débit.

Vers la fin de la simulation, l'augmentation des ACK dupliqués et des retransmissions indique que la file d'attente de 15 paquets commence à saturer régulièrement.

✓ **Observations principales**

- Fenêtre de congestion variable et oscillante ;
- Pertes régulières détectées via ACK dupliqués ;
- Retransmissions rapides, mais accompagnées de baisses brutales du débit ;
- Utilisation de la bande passante correcte, mais nettement moins stable que sous BBR.

Les graphiques TCP (Stevens et tcptrace) montrent clairement les oscillations périodiques typiques de NewReno, contrastant fortement avec la courbe quasi-linéaire et stable observée avec BBR.

2.3. Conclusion sur TCP NewReno

TCP NewReno reste un algorithme fiable et largement déployé grâce à sa simplicité et sa robustesse. Dans le contexte de notre simulation (lien bottleneck de 2 Mbps, délai de propagation de 100 ms et file d'attente limitée à 15 paquets), il utilise correctement la bande passante, mais au prix d'**oscillations importantes du débit** et d'un **taux de pertes plus élevé** que celui observé avec BBR.

Sa forte réactivité aux pertes le rend robuste dans de nombreux scénarios, mais le rend également plus sensible au *bufferbloat* et moins efficace sur les liens à haut *Bandwidth-Delay Product* (BDP) ou en présence de pertes non liées à la congestion. NewReno illustre parfaitement les limites des approches purement *loss-based* dans les réseaux modernes.

3. TCP CUBIC

3.1. Qu'est-ce que TCP CUBIC ?

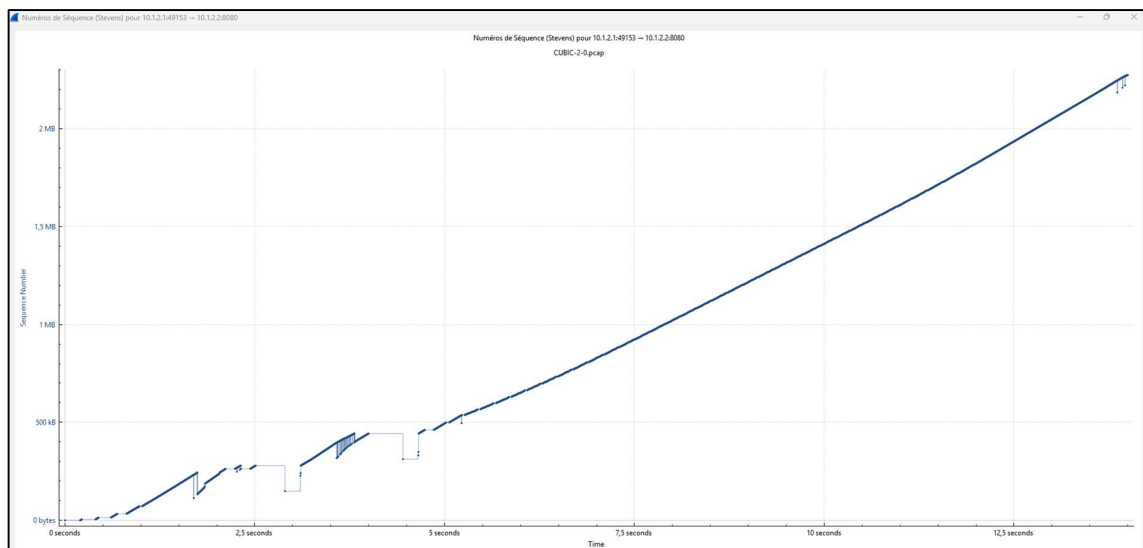
TCP CUBIC est l'algorithme de contrôle de congestion par défaut dans la plupart des noyaux Linux modernes. Il appartient à la famille des protocoles *loss-based*, mais se distingue par une fonction de croissance cubique de la fenêtre de congestion, indépendante du RTT.

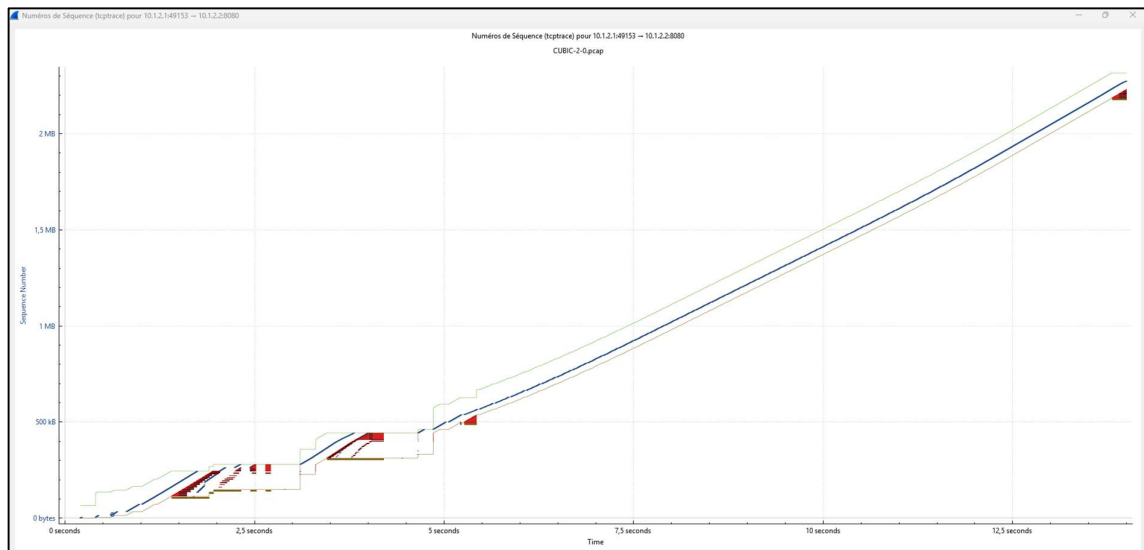
Contrairement à NewReno qui utilise une croissance additive linéaire en phase de Congestion Avoidance, CUBIC calcule la fenêtre cible selon une fonction cubique du temps écoulé depuis la dernière perte. Cela lui permet :

- Une croissance rapide lorsque la fenêtre est loin de sa valeur maximale précédente ;
- Une croissance plus lente et plus stable lorsque la fenêtre se rapproche de la valeur optimale ;
- Une meilleure scalabilité sur les liens à large bande passante et à haut *Bandwidth-Delay Product* (BDP).

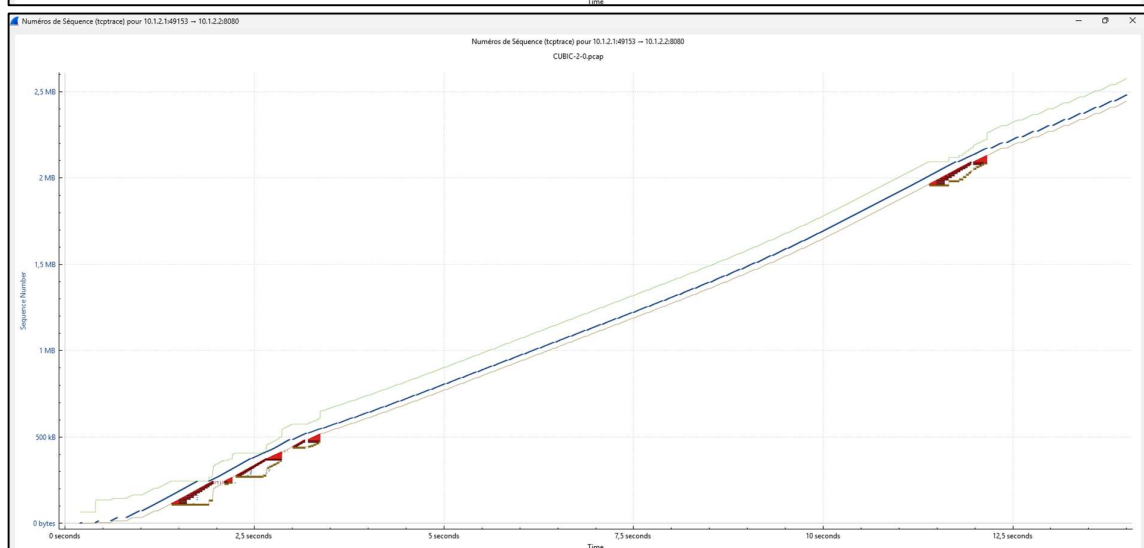
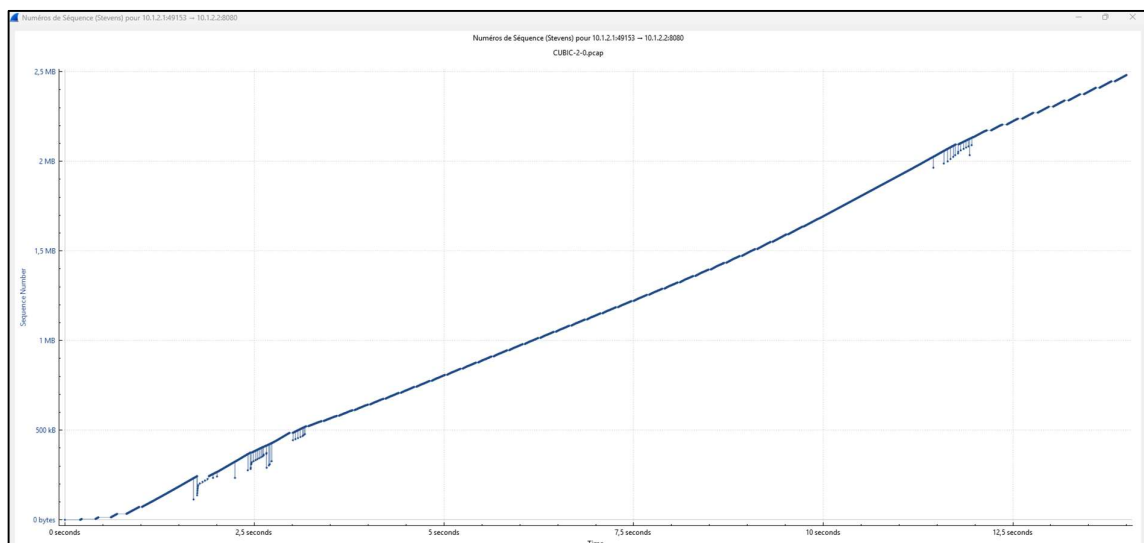
CUBIC est conçu pour être plus agressif que NewReno sur les réseaux à forte capacité, tout en restant équitable avec les autres flux TCP.

3.2. Analyse de performance de TCP CUBIC





Figs: Capture WireShark, Graphes TCP Cubic(Stevens et tcptrace) Machine AGBENONZAN



Figs: Capture WireShark, Graphes TCP Cubic(Stevens et tcptrace) Machine SANOGO

L'analyse des captures PCAP de TCP CUBIC (machines SANOGO et AGBENONZAN) met en évidence un comportement **intermédiaire** entre NewReno et BBR.

- **Établissement de la connexion**

Le *handshake* TCP est identique aux deux autres protocoles : la fenêtre annoncée passe de 65 535 à **131 072 octets** dès le troisième paquet. Comme pour NewReno, cette valeur n'est pas fixe et évolue au cours de la simulation.

- **Phase initiale (Slow Start)**

Les premiers paquets montrent une croissance rapide du numéro de séquence, caractéristique de la phase Slow Start. CUBIC double la fenêtre à chaque RTT, ce qui se traduit par un envoi soutenu au début de la connexion.

- **Phase de régime stable (Congestion Avoidance)**

Après la phase de démarrage, CUBIC passe en mode Congestion Avoidance. La courbe du numéro de séquence présente une croissance plus régulière et plus agressive que celle de NewReno. Grâce à sa fonction cubique, CUBIC récupère plus rapidement après une perte : la fenêtre augmente rapidement lorsqu'elle est loin du point de congestion précédent, puis ralentit en s'en approchant.

On observe des ACK dupliqués et des **Fast Retransmissions**, indiquant des pertes de paquets. À chaque perte détectée, CUBIC réduit la fenêtre, puis reprend sa croissance cubique. Cela se traduit par des oscillations du débit, moins prononcées que chez NewReno, avec une meilleure récupération.

Le comportement observé sur la capture de la machine AGBENONZAN est similaire, confirmant ainsi la **reproductibilité** des résultats malgré les différences d'environnement (WSL2 versus MSYS2).

✓ **Observations principales**

- Fenêtre de congestion variable et dynamique ;
- Croissance cubique permettant une meilleure utilisation de la bande passante après les pertes ;
- Taux de pertes intermédiaire entre NewReno et BBR ;
- Meilleure scalabilité que NewReno grâce à l'indépendance vis-à-vis du RTT.

Les graphiques TCP montrent une courbe de numéro de séquence plus lisse que celle de NewReno, avec des oscillations moins brutales, mais toujours présentes contrairement à la stabilité quasi-linéaire observée avec BBR.

3.3. Conclusion sur TCP CUBIC

TCP CUBIC représente une évolution significative des algorithmes *loss-based* traditionnels. Dans le cadre de notre simulation (lien bottleneck de 2 Mbps, délai de propagation de 100 ms et file d'attente limitée à 15 paquets), il offre un excellent compromis entre la simplicité de NewReno et les performances de BBR. Grâce à sa fonction de croissance cubique, il permet une **meilleure récupération après perte** et une utilisation plus efficace de la bande passante.

Cependant, comme tous les protocoles basés sur les pertes, CUBIC reste sensible aux pertes non liées à la congestion et génère des oscillations du débit, bien que nettement moins marquées que chez NewReno. Il est particulièrement bien adapté aux réseaux à large bande passante et à haut *Bandwidth-Delay Product* (BDP), ce qui explique son adoption massive en tant qu'algorithme par défaut dans la plupart des noyaux Linux modernes.

4. Analyse Comparative des Protocoles TCP

Après l'étude individuelle des trois variantes TCP (NewReno, CUBIC et BBR) dans le scénario simulé avec NS-3.40 (lien bottleneck de 2 Mbps, délai de propagation de 100 ms, file d'attente limitée à 15 paquets, durée de 15 secondes et trois flux concurrents via OnOffHelper), cette section propose une synthèse quantitative des performances observées.

Protocole	Debit_Mbps	Pertes	TxPackets	RxPackets
NEWRENO	1.143	9	3449	3404
CUBIC	1.48663	61	4538	4438
BBR	1.43794	19	4346	4282

Données extraite de `resultats_comparaison.csv`

4.1. Tableau comparatif des métriques principale

Critère	TCP NewReno	TCP CUBIC	TCP BBR	Commentaire
Débit moyen	1,143 Mbps	1,487 Mbps	1,438 Mbps	CUBIC légèrement supérieur
Approche de contrôle de congestion	<i>Loss-based</i>	<i>Loss-based</i> (cubique)	<i>Model-based</i>	BBR est proactif
Fenêtre de congestion	Variable (dents de scie)	Variable (récupération rapide)	Constante (~131072 octets)	Marqueur fort de BBR
Stabilité du débit	Faible (oscillations marquées)	Moyenne	Élevée (quasi-linéaire)	BBR le plus stable
Nombre de pertes	9	61	19	NewReno le plus robuste sur ce point
Efficacité (Rx/Tx)	98,70 %	97,80 %	98,53 %	Très bonne efficacité globale
Latence / Bufferbloat	Plus élevée	Moyenne	Faible	BBR limite l'inflight
Comportement en phase initiale	Slow Start agressif	Slow Start agressif	Startup + Drain rapide	BBR remplit le pipe plus efficacement
Reproductibilité (2 machines)	Bonne	Bonne	Excellente	Comportement très similaire

4.2. Synthèse et interprétation des différences

L'observation visuelle des graphes Stevens et tcptrace issus des captures PCAP permet de confirmer les résultats quantitatifs :

- TCP NewReno : les graphes montrent clairement des oscillations en dents de scie très marquées (pentes linéaires entrecoupées de chutes brutales). On voit sur le graphique du numéro de séquence une croissance régulière mais saccadée, typique d'une réaction forte à chaque perte (réduction de fenêtre par 2). Cela explique son débit plus faible (1,143 Mbps) et sa sensibilité au *bufferbloat*.
- TCP CUBIC : les graphes révèlent une courbe plus lisse et plus agressive que NewReno. La fonction cubique se traduit visuellement par une récupération beaucoup plus rapide après chaque perte. On observe moins de dents de scie et une pente plus forte en phase de reprise, ce qui lui permet d'atteindre le meilleur débit (1,487 Mbps).
- TCP BBR : les graphes Stevens montrent une ligne quasi-parfaite et quasi-linéaire dès la fin de la phase Startup. Contrairement aux deux autres, on ne voit presque aucune oscillation après 3 secondes. Cela s'explique par son approche *model-based* : il estime en continu le BtlBw et le RTprop et maintient la fenêtre proche de $2 \times \text{BDP}$, évitant ainsi les pertes inutiles liées à la congestion.

Équité (Fairness) : Avec seulement trois flux, BBR reste relativement équitable. Cependant, sa phase Startup agressive et son comportement *model-based* peuvent le rendre plus agressif face à des protocoles *loss-based* dans des scénarios avec un plus grand nombre de flux concurrents.

4.3. Limites

Cette simulation présente plusieurs limites :

- Durée courte (15 secondes seulement), ce qui ne permet pas d'observer le comportement à long terme (effet de la phase ProbeRTT de BBR par exemple).
- Environnement simplifié : un seul lien *bottleneck*, pas de pertes aléatoires, pas de trafic cross-traffic réel.
- Absence de mesure fine de la latence bout-en-bout et du *jitter*.
- Équité évaluée sur seulement trois flux ; un nombre plus élevé aurait été plus représentatif des réseaux réels.

Malgré ces limites, les résultats restent cohérents et reproductibles sur deux environnements distincts.

4.4. Recommandations

- Réseaux à faible latence / faible perte → **NewReno** ou **CUBIC**.
- Réseaux avec latence élevée, haut BDP ou tampon limité → **BBR** est supérieur.
- Environnements multi-flux concurrents → attention à l'équité (BBR peut être plus agressif).

CONCLUSION GÉNÉRALE

Ce projet de simulation sous NS-3.40 avait pour objectif de comparer les performances de trois variantes majeures du protocole TCP — TCP NewReno, TCP CUBIC et TCP BBR — dans un environnement contraint représentatif des réseaux à latence non négligeable : un lien *bottleneck* de 2 Mbps, un délai de propagation de 100 ms, une file d’attente limitée à 15 paquets et trois flux concurrents générés via *OnOffHelper* sur une durée de 15 secondes.

L’analyse détaillée des captures PCAP sur deux machines différentes (AGBENONZAN sous WSL2 Ubuntu et SANOGO sous MSYS2 MINGW64), complétée par les résultats de FlowMonitor, a permis de mettre en évidence des comportements distincts :

- **TCP NewReno** : algorithme historique *loss-based*, réactif aux pertes, caractérisé par des oscillations marquées du débit et un taux de pertes élevé. Robuste et simple, il montre cependant ses limites dans les scénarios à BDP modéré où la file d’attente se sature rapidement.
- **TCP CUBIC** : amélioration notable de NewReno grâce à sa fonction de croissance cubique indépendante du RTT. Il récupère plus rapidement après une perte, exploite mieux la bande passante et présente des oscillations moins brutales. CUBIC est particulièrement adapté aux réseaux modernes à haut débit.
- **TCP BBR** : approche proactive *model-based*. En estimant continuellement la bande passante du goulot ($BtlBw$) et le délai minimal (RT_{prop}), il maintient une fenêtre quasi-constante et opère au plus près du *Bandwidth-Delay Product* (BDP). Les captures révèlent une stabilité exceptionnelle du débit et une réduction significative du *bufferbloat*. BBR s’est imposé comme le plus performant dans notre simulation.

Ces résultats confirment que le choix de l’algorithme de contrôle de congestion doit être adapté aux caractéristiques du réseau : bande passante, latence, taille des tampons et niveau de perte. Dans un environnement à délai élevé et tampon limité, **BBR offre un avantage clair** en termes de stabilité, d’efficacité et de maîtrise de la latence, tandis que NewReno et CUBIC restent pertinents dans d’autres contextes, notamment pour l’équité et la compatibilité.

Ce projet a également validé la portabilité de NS-3.40 sur différentes plateformes (WSL2 et MSYS2) ainsi que l’intérêt des outils d’analyse tels que Wireshark pour comprendre finement le comportement des protocoles de transport.

Perspectives

Il serait intéressant d'étendre cette étude en :

- Augmentant le nombre de flux concurrents afin de mieux évaluer l'équité entre BBR et les protocoles *loss-based* ;
- Testant différents scénarios (haut débit, pertes aléatoires, liens satellite) ;
- Intégrant des métriques d'équité telles que l'indice de Jain (*Jain's Fairness Index*) et le *throughput* agrégé.

En conclusion, cette simulation illustre l'évolution des mécanismes de contrôle de congestion TCP et souligne l'importance d'adapter ces algorithmes aux contraintes spécifiques des réseaux modernes.

Annexes

Liste des sigles et abréviations

Sigle / Abréviation	Signification complète
BBR	Bottleneck Bandwidth and Round-Trip Time
BDP	Bandwidth-Delay Product
BtlBw	Bottleneck Bandwidth
NS-3	Network Simulator 3
PCAP	Packet Capture
RTT	Round-Trip Time
RTprop	Round-Trip propagation time (minimal)
TCP	Transmission Control Protocol
WSL2	Windows Subsystem for Linux 2
SYN	Synchronize (paquet TCP)
ACK	Acknowledgment
SACK	Selective Acknowledgment
Fast Retransmit	Retransmission rapide
Fast Recovery	Récupération rapide
OnOffHelper	Application de trafic On/Off (NS-3)

Liste des mots techniques et termes compliqués

Terme technique	Explication simple / Définition
Algorithme <i>loss-based</i>	Algorithme qui réagit aux pertes de paquets pour détecter la congestion
Algorithme <i>model-based</i>	Algorithme qui modélise le réseau (bande passante + délai) au lieu de réagir aux pertes
<i>Bandwidth-Delay Product</i> (BDP)	Quantité de données qui peuvent être en vol sur le lien (bande passante $\times$ délai)
<i>Bufferbloat</i>	Engorgement des files d'attente qui augmente fortement la latence
<i>Congestion Avoidance</i>	Phase où la fenêtre de congestion augmente lentement et linéairement
<i>Fast Retransmit</i>	Retransmission immédiate dès 3 ACK dupliqués
<i>Partial ACK</i>	ACK qui reconnaît seulement une partie des données perdues
<i>Pacing rate</i>	Rythme d'envoi des paquets (contrôle fin du débit)
<i>ProbeBW</i>	Phase de BBR où l'on sonde la bande passante disponible
<i>ProbeRTT</i>	Phase de BBR où l'on mesure le délai minimal
<i>Slow Start</i>	Phase initiale où la fenêtre double à chaque RTT
<i>Startup</i> (BBR)	Phase initiale très agressive de BBR
<i>Drain</i> (BBR)	Phase où BBR vide les files d'attente accumulées
<i>In-flight</i>	Données envoyées mais pas encore acquittées
Goulot d'étranglement	Point le plus étroit du chemin réseau (bottleneck)
Scalabilité	Capacité à bien fonctionner sur des liens à très haut débit
Équité (fairness)	Capacité des flux à partager équitablement la bande passante
Jain's Fairness Index	Métrique mathématique pour mesurer l'équité entre plusieurs flux

Contenue du Script de configuration et récupération du Trafic TCP

Tcp-comparaison.cc

```
#include "ns3/core-module.h"
#include "ns3/network-module.h"
#include "ns3/internet-module.h"
#include "ns3/point-to-point-module.h"
#include "ns3/applications-module.h"
#include "ns3/flow-monitor-module.h"
#include <fstream>
#include <iomanip>
using namespace ns3;

NS_LOG_COMPONENT_DEFINE ("ComparaisonTcpFinal");
int main (int argc, char *argv[])
{
    CommandLine cmd;
    cmd.Parse (argc, argv);

    // 1. Création des nœuds
    NodeContainer nNR, nCB, nBR;
    nNR.Create (2); nCB.Create (2); nBR.Create (2);

    // 2. Configuration du lien
    PointToPointHelper p2p;
    p2p.SetDeviceAttribute ("DataRate", StringValue ("2Mbps"));
    p2p.SetChannelAttribute ("Delay", StringValue ("100ms"));
    p2p.SetQueue ("ns3::DropTailQueue", "MaxSize", StringValue ("15p"));

    NetDeviceContainer dNR = p2p.Install (nNR);
    NetDeviceContainer dCB = p2p.Install (nCB);
    NetDeviceContainer dBR = p2p.Install (nBR);

    // 3. Pile Internet
    InternetStackHelper stack;
    stack.InstallAll ();

    // 4. IPs
    Ipv4AddressHelper address;
    Ipv4InterfaceContainer iNR, iCB, iBR;
    address.SetBase ("10.1.1.0", "255.255.255.0"); iNR = address.Assign (dNR);
    address.SetBase ("10.1.2.0", "255.255.255.0"); iCB = address.Assign (dCB);
    address.SetBase ("10.1.3.0", "255.255.255.0"); iBR = address.Assign (dBR);

    // 5. CONFIGURATION DES VARIANTES
```

```

    Config::Set ("/NodeList/0/$ns3::TcpL4Protocol/SocketType", TypeIdValue
(TcpNewReno::GetTypeId ());
    Config::Set ("/NodeList/2/$ns3::TcpL4Protocol/SocketType", TypeIdValue
(TcpCubic::GetTypeId ());
    Config::Set ("/NodeList/4/$ns3::TcpL4Protocol/SocketType", TypeIdValue
(TcpBbr::GetTypeId ());

    uint16_t port = 8080;

    // 6. Applications OnOff
    // NewReno
    OnOffHelper onoffNR ("ns3::TcpSocketFactory", InetSocketAddress
(iNR.GetAddress (1), port));
    onoffNR.SetAttribute ("DataRate", StringValue ("10Mbps"));
    onoffNR.SetAttribute ("PacketSize", UIntegerValue (1024));
    onoffNR.SetAttribute ("OnTime", StringValue
("ns3::ConstantRandomVariable[Constant=1]"));
    onoffNR.SetAttribute ("OffTime", StringValue
("ns3::ConstantRandomVariable[Constant=0]"));
    ApplicationContainer sourceNR = onoffNR.Install (nNR.Get (0));
    sourceNR.Start (Seconds (1.0)); sourceNR.Stop (Seconds (15.0));

    // Cubic
    OnOffHelper onoffCB ("ns3::TcpSocketFactory", InetSocketAddress
(iCB.GetAddress (1), port));
    onoffCB.SetAttribute ("DataRate", StringValue ("10Mbps"));
    onoffCB.SetAttribute ("PacketSize", UIntegerValue (1024));
    onoffCB.SetAttribute ("OnTime", StringValue
("ns3::ConstantRandomVariable[Constant=1]"));
    onoffCB.SetAttribute ("OffTime", StringValue
("ns3::ConstantRandomVariable[Constant=0]"));
    ApplicationContainer sourceCB = onoffCB.Install (nCB.Get (0));
    sourceCB.Start (Seconds (1.0)); sourceCB.Stop (Seconds (15.0));

    // BBR
    OnOffHelper onoffBR ("ns3::TcpSocketFactory", InetSocketAddress
(iBR.GetAddress (1), port));
    onoffBR.SetAttribute ("DataRate", StringValue ("10Mbps"));
    onoffBR.SetAttribute ("PacketSize", UIntegerValue (1024));
    onoffBR.SetAttribute ("OnTime", StringValue
("ns3::ConstantRandomVariable[Constant=1]"));
    onoffBR.SetAttribute ("OffTime", StringValue
("ns3::ConstantRandomVariable[Constant=0]"));
    ApplicationContainer sourceBR = onoffBR.Install (nBR.Get (0));
    sourceBR.Start (Seconds (1.0)); sourceBR.Stop (Seconds (15.0));

```

```

// 7. Récepteurs - CORRECTION ICI !
PacketSinkHelper sink ("ns3::TcpSocketFactory", InetSocketAddress
(Ipv4Address::GetAny (), port));

// NewReno sink
ApplicationContainer sinkNR = sink.Install (nNR.Get (1));
sinkNR.Start (Seconds (0.0));
sinkNR.Stop (Seconds (15.0));

// Cubic sink
ApplicationContainer sinkCB = sink.Install (nCB.Get (1));
sinkCB.Start (Seconds (0.0));
sinkCB.Stop (Seconds (15.0));

// BBR sink
ApplicationContainer sinkBR = sink.Install (nBR.Get (1));
sinkBR.Start (Seconds (0.0));
sinkBR.Stop (Seconds (15.0));

// 8. Monitor & PCAP
FlowMonitorHelper flowmon;
Ptr<FlowMonitor> monitor = flowmon.InstallAll();

p2p.EnablePcap ("NEWRENO", dNR.Get(0));
p2p.EnablePcap ("CUBIC", dCB.Get(0));
p2p.EnablePcap ("BBR", dBR.Get(0));

std::cout << "Simulation en cours... (15 secondes)" << std::endl;
Simulator::Stop (Seconds (15.0));
Simulator::Run ();

// 9. Analyse des résultats
monitor->CheckForLostPackets();
Ptr<Ipv4FlowClassifier> classifier =
DynamicCast<Ipv4FlowClassifier>(flowmon.GetClassifier());
FlowMonitor::FlowStatsContainer stats = monitor->GetFlowStats();

// Export des résultats
std::ofstream resultFile;
resultFile.open("resultats_comparaison.csv");
resultFile << "Protocole;Debit_Mbps;Pertes;TxPackets;RxPackets\n";

std::cout << "\n=== COMPARAISON DES PROTOCOLES ===\n";
std::cout << "===== \n";

```

```

for (auto &flow : stats) {
    Ipv4FlowClassifier::FiveTuple t = classifier->FindFlow(flow.first);
    if (t.destinationPort == port) {
        std::string label;
        if (t.sourceAddress == "10.1.1.1") label = "NEWRENO";
        else if (t.sourceAddress == "10.1.2.1") label = "CUBIC";
        else label = "BBR";

        double dureeUtile = 14.0; // De 1s à 15s
        double debit = flow.second.rxBytes * 8.0 / dureeUtile / 1e6;
        double txPackets = flow.second.txPackets;
        double rxPackets = flow.second.rxPackets;
        double pertes = flow.second.lostPackets;

        // Affichage console
        std::cout << std::fixed << std::setprecision(2);
        std::cout << " 📌 " << label << " (" << t.sourceAddress << ")" <<
std::endl;

        std::cout << " 📊 Débit : " << debit << " Mbps" << std::endl;
        std::cout << " 📦 Pertes : " << pertes << " paquets" << std::endl;
        std::cout << " 📡 Tx : " << txPackets << " paquets" << std::endl;
        std::cout << " 📡 Rx : " << rxPackets << " paquets" << std::endl;
        std::cout << " 📈 Efficacité: " << (rxPackets/txPackets*100) << "%" <<
std::endl;
        std::cout << "-----" << std::endl;

        // Export CSV
        resultFile << label << ";" << debit << ";" << pertes << ";"
            << txPackets << ";" << rxPackets << "\n";
    }
}
resultFile.close();
std::cout << "\n 📁 Résultats sauvegardés dans 'resultats_comparaison.csv'\n";

Simulator::Destroy ();
return 0;
}

```

Contenue du Script de lancement

run-experiment.sh

```
#!/bin/bash
```

```
echo
```

```
"=====
```

```
=="
```

```
echo "COMPARAISON DES PROTOCOLES TCP"
```

```
echo "NewReno vs CUBIC vs BBR"
```

```
echo
```

```
"=====
```

```
=="
```

```
echo ""
```

```
cd ~/ns-allinone-3.40/ns-3.40
```

```
# Nettoyer les anciens fichiers
```

```
echo "[1/4] Nettoyage des anciens fichiers..."
```

```
rm -f ../*.pcap ../NEWRENO*.pcap ../CUBIC*.pcap ../BBR*.pcap
```

```
rm -f scratch/comparison-tcp/resultats_comparaison.csv
```

```
# Compiler
```

```
echo "[2/4] Compilation en cours..."
```

```
./ns3 build scratch/comparison-tcp/tcp-comparison
```

```
# Exécuter la simulation
```

```
echo "[3/4] Exécution des simulations..."
```

```
./ns3 run scratch/comparison-tcp/tcp-comparison
```

```
# Afficher les résultats
```

```
echo "[4/4] Resultats obtenus:"
```

```
cd scratch/comparison-tcp
```

```
if [ -f "resultats_comparaison.csv" ]; then
```

```
    echo ""
```

```
    echo "=== CONTENU DE resultats_comparaison.csv ==="
```

```
    cat resultats_comparaison.csv
```

```
    echo
```

```
"=====
```

```
    echo ""
```

```
    echo "Fichiers PCAP disponibles dans le repertoire principal:"
```

```
    ls -la ~/ns-allinone-3.40/ns-3.40/*.pcap 2>/dev/null | | echo "  Aucun  
fichier PCAP trouve"
```

```
else
```

```
    echo "ERREUR: Fichier resultats_comparaison.csv non trouve"
```

fi

echo ""

echo

"=====

=="

echo "EXPÉRIENCE TERMINÉE"

echo

"=====

=="